Azure Data Engineer

End-to-End Complete Study Guide

Every Skill · Every Concept · Definitions · Examples · Code

Certifications: DP-900 | DP-203 | DP-700

Table of Contents

1. SQL — Complete Deep Dive

1.1 Foundations

SELECT, FROM, WHERE

The core of every SQL query. SELECT picks columns, FROM specifies the table, WHERE filters rows.

Example:

Get all employees in the Sales department with salary above 50000.

Code:

```
SELECT name, salary
FROM employees
WHERE department = 'Sales' AND salary > 50000;
```

ORDER BY, LIMIT / TOP

ORDER BY sorts results (ASC default, DESC for reverse). LIMIT (MySQL/PostgreSQL) or TOP (SQL Server) restricts the number of rows returned.

Code:

```
SELECT name, salary FROM employees
ORDER BY salary DESC
LIMIT 10; -- top 10 earners
```

DISTINCT

Returns only unique values in a column or combination of columns.

Code:

```
SELECT DISTINCT department FROM employees;
SELECT DISTINCT department, job_title FROM employees;
```

Aliases (AS)

Rename columns or tables within a query for readability or to avoid naming conflicts.

Code:

```
SELECT e.name AS employee_name, d.name AS dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.id;
```

1.2 Filtering & Conditions

BETWEEN, IN, LIKE, IS NULL

Operators for range checks, membership tests, pattern matching, and null checks.

Code:

```
-- BETWEEN (inclusive)
SELECT * FROM orders WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31';

-- IN
SELECT * FROM employees WHERE department IN ('HR','Finance','IT');

-- LIKE (% = any chars, _ = one char)
SELECT * FROM customers WHERE email LIKE '%@gmail.com';

-- IS NULL / IS NOT NULL
SELECT * FROM employees WHERE manager_id IS NULL;
```

CASE WHEN

Adds conditional logic inside a query — similar to if-else. Can be used in SELECT, WHERE, ORDER BY.

Code:

```
SELECT name, salary,
CASE
WHEN salary >= 100000 THEN 'High'
WHEN salary >= 60000 THEN 'Mid'
ELSE 'Low'
END AS salary_band
FROM employees;
```

1.3 Aggregations & GROUP BY

Aggregate Functions

COUNT, SUM, AVG, MIN, MAX — operate on a set of rows and return a single value per group.

Code:

```
SELECT department,
COUNT(*) AS headcount,
AVG(salary) AS avg_salary,
MAX(salary) AS max_salary,
SUM(salary) AS salary_budget
FROM employees
GROUP BY department;
```

HAVING

Filters groups after GROUP BY (like WHERE but for aggregated results).

Example:

Find departments with more than 10 employees.

Code:

```
SELECT department, COUNT(*) AS cnt
FROM employees
GROUP BY department
HAVING COUNT(*) > 10;
```

1.4 JOINs

INNER JOIN

Returns rows where the join condition matches in BOTH tables.

Code:

```
SELECT e.name, d.name AS dept
FROM employees e
INNER JOIN departments d ON e.dept_id = d.id;
```

LEFT / RIGHT / FULL OUTER JOIN

LEFT JOIN: all rows from left + matching from right (NULLs if no match). RIGHT is opposite. FULL returns all rows from both.

Code:

```
-- LEFT JOIN: keep all employees even if no department
SELECT e.name, d.name AS dept
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id;
```

```
-- FULL OUTER JOIN
SELECT e.name, d.name
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.id;
```

CROSS JOIN & SELF JOIN

CROSS JOIN produces cartesian product (every row × every row). SELF JOIN joins a table to itself.

Example:

Self join to find each employee and their manager.

Code:

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id;
```

1.5 Window Functions (Critical for DE)

ROW_NUMBER, RANK, DENSE_RANK

Assign sequential numbers or ranks to rows within a partition. RANK skips numbers on ties; DENSE_RANK does not.

Code:

```
SELECT name, department, salary,
ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num,
RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk,
DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dense_rnk
FROM employees;
```

LAG and LEAD

Access the value of a previous row (LAG) or next row (LEAD) within a partition — useful for calculating differences between consecutive rows.

Example:

Calculate month-over-month revenue change.

Code:

```
SELECT month, revenue,
LAG(revenue) OVER (ORDER BY month) AS prev_month_rev,
revenue - LAG(revenue) OVER (ORDER BY month) AS mom_change,
LEAD(revenue) OVER (ORDER BY month) AS next_month_rev
FROM monthly_sales;
```

SUM / AVG / COUNT as Window Functions

Aggregate functions used with OVER() to compute running totals, moving averages, or cumulative counts without collapsing rows.

Code:

```
SELECT order_date, amount,
SUM(amount) OVER (ORDER BY order_date
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
AVG(amount) OVER (ORDER BY order_date
ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS moving_avg_7d
FROM orders;
```

NTILE, PERCENT_RANK, CUME_DIST

Distribution functions. NTILE(n) splits rows into n equal buckets. PERCENT_RANK and CUME_DIST give relative rank as a percentage.

Code:

```
SELECT name, salary,
NTILE(4) OVER (ORDER BY salary) AS quartile,
PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank,
CUME_DIST() OVER (ORDER BY salary) AS cume_dist
FROM employees;
```

1.6 CTEs & Subqueries

Common Table Expressions (CTE)

Named temporary result sets using WITH clause. Improve readability and allow referencing the same subquery multiple times.

Code:

```
WITH dept_avg AS (
SELECT department, AVG(salary) AS avg_sal
FROM employees GROUP BY department
),
high_earners AS (
SELECT e.name, e.salary, d.avg_sal
FROM employees e
JOIN dept_avg d ON e.department = d.department
WHERE e.salary > d.avg_sal
)
SELECT * FROM high_earners ORDER BY salary DESC;
```

Recursive CTE

A CTE that references itself — used to traverse hierarchical data like org charts or bill-of-materials.

Example:

Traverse employee hierarchy from CEO down.

Code:

```
WITH RECURSIVE org_chart AS (
SELECT id, name, manager_id, 1 AS level
FROM employees WHERE manager_id IS NULL -- anchor
UNION ALL
SELECT e.id, e.name, e.manager_id, oc.level + 1
FROM employees e
JOIN org_chart oc ON e.manager_id = oc.id -- recursive
)
SELECT name, level FROM org_chart ORDER BY level;
```

1.7 Indexing & Query Optimization

Clustered vs Non-Clustered Index

Clustered index physically sorts the table by the index key (one per table). Non-clustered index is a separate structure with pointers to data rows (multiple allowed).

Code:

```
-- Create non-clustered index on frequently filtered column
CREATE INDEX idx_emp_dept ON employees(department);
```

```
-- Composite index (covers both filter and sort)
CREATE INDEX idx_order_customer_date ON orders(customer_id, order_date DESC);
```

Query Execution Plan

A visual/textual breakdown of how the SQL engine executes a query — reveals table scans, index seeks, sort operations, and estimated costs.

Code:

```
-- SQL Server
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
SELECT * FROM orders WHERE customer_id = 1001;
```

```
-- PostgreSQL
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 1001;
```

Partitioning

Divides a large table into smaller physical segments based on a column (e.g. date). Queries that filter on the partition key only scan relevant partitions (partition pruning).

Code:

```
-- SQL Server: partition by year
CREATE PARTITION FUNCTION pf_year(int) AS RANGE RIGHT
FOR VALUES (2021, 2022, 2023, 2024);
```

1.8 Advanced SQL Concepts

PIVOT and UNPIVOT

PIVOT rotates rows into columns. UNPIVOT does the reverse.

Code:

```
-- PIVOT: show sales per product as columns
SELECT * FROM (
SELECT product, month, sales FROM monthly_sales
) src
PIVOT (SUM(sales) FOR month IN ([Jan],[Feb],[Mar])) pvt;
```

MERGE Statement (Upsert)

Combines INSERT, UPDATE, and DELETE in one statement based on a matching condition. Used for SCD Type 1/2 and CDC operations.

Code:

```
MERGE target_customers AS tgt
USING source_updates AS src ON tgt.id = src.id
WHEN MATCHED THEN
UPDATE SET tgt.email = src.email, tgt.city = src.city
WHEN NOT MATCHED BY TARGET THEN
INSERT (id, name, email, city) VALUES (src.id, src.name, src.email, src.city)
WHEN NOT MATCHED BY SOURCE THEN
DELETE;
```

STRING_AGG / LISTAGG

Aggregates multiple row values into a single delimited string.

Code:

```
SELECT department,
STRING_AGG(name, ', ') AS employees
FROM employees
GROUP BY department;
```

2. Python for Data Engineering

2.1 Pandas — Core Operations

Reading & Writing Data

Pandas supports reading from CSV, Parquet, Excel, JSON, SQL, and many other formats.

Code:

```
import pandas as pd

df = pd.read_csv("data.csv")
df = pd.read_parquet("data.parquet")
df = pd.read_json("data.json")
df = pd.read_excel("data.xlsx", sheet_name="Sheet1")

# Write
df.to_parquet("out.parquet", index=False)
df.to_csv("out.csv", index=False)
```

DataFrame Operations

Core operations for inspecting, filtering, selecting, and transforming DataFrames.

Code:

```
df.shape # (rows, cols)
df.dtypes # column data types
df.describe() # stats summary
df.info() # memory, nulls

# Select
df[['col1','col2']]
df.loc[df['age'] > 30, ['name','age']]

# Filter
df[df['city'] == 'NYC']
df.query("salary > 50000 and dept == 'IT'")
```

GroupBy & Aggregations

Group rows by one or more columns and apply aggregate functions.

Code:

```
# Simple groupby
df.groupby('department')['salary'].mean()

# Multiple aggregations
df.groupby('department').agg(
headcount=('name','count'),
avg_sal=('salary','mean'),
max_sal=('salary','max')
).reset_index()
```

Merge / Join DataFrames

Combine two DataFrames on a key — similar to SQL JOINs.

Code:

```
result = pd.merge(df_orders, df_customers,
on='customer_id', how='left')

# Multiple keys
pd.merge(df1, df2, on=['year','month'], how='inner')
```

Handling Missing Data

Identify, drop, or fill missing values.

Code:

```
df.isnull().sum() # count nulls per column
df.dropna(subset=['email']) # drop rows missing email
df['age'].fillna(df['age'].mean()) # fill with mean
df.fillna({'city': 'Unknown', 'score': 0})
```

Apply & Lambda

Apply a custom function to each row or column.

Code:

```
# Apply to column
df['name_upper'] = df['name'].apply(lambda x: x.upper())

# Apply row-wise
df['full_name'] = df.apply(lambda r: r['first'] + ' ' + r['last'], axis=1)

# Vectorised string ops (faster)
df['email_domain'] = df['email'].str.split('@').str[1]
```

Pivot Tables & Crosstabs

Code:

```
# Pivot
df.pivot_table(values='revenue', index='region',
columns='quarter', aggfunc='sum', fill_value=0)

# Crosstab
pd.crosstab(df['gender'], df['department'])
```

2.2 Python & Azure SDK

Azure Storage SDK — ADLS Gen2

Use azure-storage-file-datalake to interact with ADLS Gen2 — upload, download, list files.

Code:

```
from azure.storage.filedatalake import DataLakeServiceClient
from azure.identity import DefaultAzureCredential

svc = DataLakeServiceClient(
account_url="https://mystore.dfs.core.windows.net",
credential=DefaultAzureCredential()
)
fs = svc.get_file_system_client("silver")
fc = fs.get_file_client("processed/orders.parquet")

# Upload
with open("orders.parquet","rb") as f:
fc.upload_data(f, overwrite=True)
```

Trigger ADF Pipeline via SDK

Code:

```
from azure.mgmt.datafactory import DataFactoryManagementClient
from azure.identity import DefaultAzureCredential

cred = DefaultAzureCredential()
client = DataFactoryManagementClient(cred, "")
run = client.pipelines.create_run(
"", "", "",
parameters={"env": "prod"})
print(f"Run ID: {run.run_id}")
```

Azure Key Vault Secrets in Python

Code:

```
from azure.keyvault.secrets import SecretClient
from azure.identity import DefaultAzureCredential

client = SecretClient(
vault_url="https://myvault.vault.azure.net",
credential=DefaultAzureCredential()
)
secret = client.get_secret("db-password")
print(secret.value)
```

2.3 Python Best Practices for DE

Logging & Error Handling

Code:

```
import logging
logging.basicConfig(level=logging.INFO,
format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger(__name__)

try:
df = pd.read_parquet("data.parquet")
log.info(f"Loaded {len(df)} rows")
except FileNotFoundError as e:
log.error(f"File not found: {e}")
raise
```

Environment Variables & Config

Never hardcode credentials. Use environment variables or Azure Key Vault.

Code:

```
import os
from dotenv import load_dotenv

load_dotenv() # reads .env file
conn_str = os.getenv("AZURE_STORAGE_CONNECTION_STRING")
db_host = os.getenv("DB_HOST", "localhost")
```

3. PySpark & Apache Spark — Complete Deep Dive

3.1 Spark Architecture

Driver & Executor Model

The Driver runs the main program, plans the DAG of tasks, and coordinates work. Executors are JVM processes on worker nodes that execute tasks and cache data. Each partition of data is processed by one task on one executor.

Code:

```
Driver Program (SparkContext / SparkSession)
|
+-- Cluster Manager (YARN / Kubernetes / Standalone)
|
+-- Worker Node 1 --> Executor (cores=4, mem=8GB)
| Task Task Task Task
+-- Worker Node 2 --> Executor (cores=4, mem=8GB)
Task Task Task Task
```

SparkSession — Entry Point

SparkSession is the unified entry point for all Spark functionality in Spark 2+.

Code:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
.appName("MyDataPipeline") \
.config("spark.sql.shuffle.partitions", "200") \
.config("spark.executor.memory", "4g") \
.config("spark.executor.cores", "4") \
.getOrCreate()

sc = spark.sparkContext # access underlying SparkContext
spark.stop() # stop session when done
```

DAG — Directed Acyclic Graph

Spark builds a DAG of transformations before executing. Each action triggers DAG compilation into stages. Stages are split at shuffle boundaries. Stages are divided into tasks (1 per partition).

Code:

```
-- DAG Stages:
Stage 1: read file -> filter -> select (no shuffle)
[SHUFFLE BOUNDARY: groupBy]
Stage 2: aggregate -> sort (after shuffle)

-- View DAG in Spark UI: http://localhost:4040
```

RDDs — Resilient Distributed Datasets

Low-level distributed collection. Immutable, partitioned, fault-tolerant. Generally prefer DataFrame API (higher level, optimized by Catalyst).

Code:

```
rdd = sc.parallelize([1, 2, 3, 4, 5], numSlices=4)
rdd2 = rdd.map(lambda x: x * 2).filter(lambda x: x > 4)
result = rdd2.collect() # action: brings data to driver
```

```
# From file
rdd = sc.textFile("hdfs://path/data.txt")
words = rdd.flatMap(lambda line: line.split(" "))
counts = words.map(lambda w: (w, 1)).reduceByKey(lambda a,b: a+b)
```

3.2 DataFrames & Datasets

Creating DataFrames

DataFrames are the primary abstraction — distributed tables with named columns and schema.

Code:

```
# From list
df = spark.createDataFrame(
[(1,"Alice",30),(2,"Bob",25)],
schema=["id","name","age"]
)
```

```
# From file
df = spark.read.csv("data.csv", header=True, inferSchema=True)
df = spark.read.parquet("data.parquet")
df = spark.read.json("data.json")
df = spark.read.format("delta").load("delta/table/")
```

```
# From ADLS Gen2
df = spark.read.parquet(
"abfss://silver@mystorage.dfs.core.windows.net/orders/")
```

Schema — StructType

Define an explicit schema to avoid inferSchema overhead and ensure correct types.

Code:

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType,
TimestampType
```

```
schema = StructType([
StructField("id", IntegerType(), nullable=False),
StructField("name", StringType(), nullable=True),
StructField("salary", DoubleType(), nullable=True),
StructField("hire_date", TimestampType(), nullable=True),
])
```

```
df = spark.read.csv("employees.csv", header=True, schema=schema)
df.printSchema()
```

3.3 Transformations — Narrow

select, selectExpr, withColumn

Select specific columns, add computed columns, or use SQL expressions.

Code:

```
from pyspark.sql.functions import col, lit, upper, concat
df.select("name", "age")
df.select(col("name"), col("salary") * 1.1)
# selectExpr — use SQL string expressions
df.selectExpr("name", "salary * 1.1 AS salary_adjusted", "age > 30 AS is_senior")
# withColumn — add or replace a column
df.withColumn("salary_adj", col("salary") * 1.1) \
.withColumn("name_upper", upper(col("name"))) \
.withColumn("source", lit("HR_SYSTEM"))
```

filter / where

Filter rows using column expressions or SQL strings — both filter and where are aliases.

Code:

```
df.filter(col("age") > 30)
df.where("age > 30 AND department = 'IT'")
df.filter((col("salary") >= 50000) & (col("active") == True))
df.filter(col("city").isin("NYC","LA","Chicago"))
df.filter(col("email").isNotNull())
df.filter(col("name").like("%Smith%"))
```

drop, dropDuplicates, distinct

Code:

```
df.drop("temp_col", "internal_id")
# Drop duplicate rows
df.distinct()
df.dropDuplicates()
df.dropDuplicates(["email"]) # dedupe on specific columns
# Drop rows with nulls
df.dropna() # any null
df.dropna(subset=["email","phone"]) # nulls in specific cols
```

rename — withColumnRenamed

Code:

```
df.withColumnRenamed("old_name", "new_name")
# Rename multiple at once using a mapping
rename_map = {"emp_id": "id", "emp_name": "name"}
for old, new in rename_map.items():
df = df.withColumnRenamed(old, new)
```

cast — Type Casting

Convert a column to a different data type.

Code:

```
from pyspark.sql.types import IntegerType, DoubleType, TimestampType
from pyspark.sql.functions import to_timestamp
df.withColumn("age", col("age").cast(IntegerType())) \
.withColumn("salary", col("salary").cast(DoubleType())) \
.withColumn("ts", to_timestamp(col("event_time"), "yyyy-MM-dd HH:mm:ss"))
```

3.4 Transformations — Wide (Shuffle)

groupBy & agg

Groups rows by one or more columns and applies aggregation functions. Triggers a shuffle.

Code:

```
from pyspark.sql.functions import count, sum, avg, max, min, countDistinct, collect_list
df.groupBy("department") \
.agg(
count("*").alias("headcount"),
avg("salary").alias("avg_salary"),
max("salary").alias("max_salary"),
countDistinct("city").alias("unique_cities"),
collect_list("name").alias("all_names")
)
```

orderBy / sort

Sorts the entire DataFrame — triggers a shuffle and a final merge. Use sparingly on large data.

Code:

```
from pyspark.sql.functions import desc, asc
df.orderBy("salary") # ascending (default)
df.orderBy(col("salary").desc()) # descending
df.orderBy(desc("dept"), asc("salary")) # multi-column sort
df.sort(col("hire_date").desc_nulls_last()) # nulls last
```

Joins

Join two DataFrames on a condition. Join type: inner, left, right, full, leftsemi, leftanti, cross.

Code:

```
# Inner join
orders.join(customers, orders.customer_id == customers.id, "inner")
# Left join
employees.join(departments, "dept_id", "left")
# Multiple conditions
df1.join(df2, (df1.id == df2.id) & (df1.year == df2.year), "inner")
# Semi join (keep left rows that have a match, no right columns)
employees.join(managers, "id", "leftsemi")
# Anti join (keep left rows with NO match)
employees.join(terminated, "id", "leftanti")
```

union & unionByName

Stack two DataFrames vertically. unionByName matches columns by name rather than position.

Code:

```
df_2023.union(df_2024) # by position
df_2023.unionByName(df_2024) # by name
df_2023.unionByName(df_2024, allowMissingColumns=True) # fills missing with null
```

pivot

Rotate rows into columns — like SQL PIVOT.

Code:

```
df.groupBy("region") \
.pivot("quarter", ["Q1","Q2","Q3","Q4"]) \
.agg(sum("revenue"))
```

3.5 Built-in Functions

String Functions

Code:

```
from pyspark.sql.functions import (
upper, lower, trim, ltrim, rtrim, length,
substring, concat, concat_ws, split, regexp_replace,
regexp_extract, instr, lpad, rpad, translate
)
df.withColumn("email_clean", trim(lower(col("email")))) \
.withColumn("domain", regexp_extract(col("email"), r'@(.+)', 1)) \
.withColumn("first3", substring(col("name"), 1, 3)) \
.withColumn("full_name", concat_ws(" ", col("first"), col("last"))) \
.withColumn("phone_clean", regexp_replace(col("phone"), r'[^0-9]', ''))
```

Date & Time Functions

Code:

```
from pyspark.sql.functions import (
current_date, current_timestamp,
to_date, to_timestamp, date_format,
year, month, dayofmonth, dayofweek, hour, minute,
datediff, months_between, add_months, date_add, date_sub,
trunc, date_trunc, unix_timestamp, from_unixtime
)
df.withColumn("today", current_date()) \
.withColumn("yr", year(col("hire_date"))) \
.withColumn("tenure_days", datediff(current_date(), col("hire_date"))) \
.withColumn("month_start", trunc(col("order_date"), "month")) \
.withColumn("fmt_date", date_format(col("order_date"), "dd-MM-yyyy"))
```

Numeric Functions

Code:

```
from pyspark.sql.functions import (
round, ceil, floor, abs, sqrt, pow, log,
greatest, least, coalesce, nullif, nanvl
)
df.withColumn("rounded", round(col("price"), 2)) \
.withColumn("city_clean",coalesce(col("city"), col("state"), lit("Unknown"))) \
.withColumn("max_val", greatest(col("a"), col("b"), col("c")))
```

Conditional Functions — when / otherwise

Equivalent to SQL CASE WHEN.

Code:

```
from pyspark.sql.functions import when
df.withColumn("grade",
when(col("score") >= 90, "A")
.when(col("score") >= 80, "B")
.when(col("score") >= 70, "C")
.otherwise("F")
)
```

Array & Map Functions

Code:

```
from pyspark.sql.functions import (
array, array_contains, array_distinct, array_union,
explode, explode_outer, posexplode,
size, flatten, array_sort,
create_map, map_keys, map_values, map_from_arrays
)
# Explode array column into multiple rows
df.withColumn("tag", explode(col("tags")))
# explode_outer keeps rows with null/empty arrays
df.withColumn("tag", explode_outer(col("tags")))
# posexplode gives index + value
df.select("id", posexplode(col("tags")).alias("pos","tag"))
# Array operations
df.withColumn("num_tags", size(col("tags"))) \
.withColumn("has_python", array_contains(col("skills"), "python"))
```

Struct & Nested Data Functions

Code:

```
from pyspark.sql.functions import struct, col
# Create struct
df.withColumn("address", struct(col("street"), col("city"), col("zip")))
# Access nested field
df.select(col("address.city"))
df.select("address.*") # expand all struct fields
# From JSON string column
from pyspark.sql.functions import from_json, to_json, get_json_object
schema = "STRUCT"
df.withColumn("parsed", from_json(col("json_col"), schema)) \
.select("parsed.name", "parsed.age")
# Extract single field from JSON string
df.withColumn("city", get_json_object(col("payload"), "$.address.city"))
```

3.6 Window Functions in PySpark

Window Specification

Define partition, order, and frame for window functions — equivalent to SQL OVER().

Code:

```
from pyspark.sql.window import Window
from pyspark.sql.functions import (
row_number, rank, dense_rank,
lag, lead, sum, avg, min, max,
percent_rank, ntile, cume_dist, first, last
)
# Define window
w_dept = Window.partitionBy("department").orderBy(col("salary").desc())
w_date = Window.partitionBy("store_id").orderBy("order_date")
w_running = Window.partitionBy("store_id").orderBy("order_date") \
.rowsBetween(Window.unboundedPreceding, Window.currentRow)
df.withColumn("rank", rank().over(w_dept)) \
.withColumn("row_num", row_number().over(w_dept)) \
.withColumn("prev_salary", lag("salary", 1).over(w_dept)) \
.withColumn("running_sum", sum("revenue").over(w_running)) \
.withColumn("moving_avg_7", avg("revenue").over(
Window.partitionBy("store_id").orderBy("order_date")
.rowsBetween(-6, 0)))
```

3.7 Reading & Writing Data

Reading Various Formats

Code:

```
# CSV
df = spark.read.option("header","true").option("inferSchema","true") \
.option("delimiter","|").csv("path/*.csv")
# Parquet (preferred for analytics — columnar, compressed)
df = spark.read.parquet("path/")
# JSON
df = spark.read.json("path/")
df = spark.read.option("multiLine","true").json("path/")
# Delta
df = spark.read.format("delta").load("abfss://gold@store.dfs.core.windows.net/sales/")
# JDBC — read from SQL Server
df = spark.read.format("jdbc") \
.option("url", "jdbc:sqlserver://server:1433;databaseName=mydb") \
.option("dbtable", "dbo.orders") \
.option("user", "sa") \
.option("password", dbutils.secrets.get("scope","db-pass")) \
.load()
```

Writing Various Formats

Code:

```
# Parquet with partitioning
df.write.mode("overwrite") \
.partitionBy("year","month") \
.parquet("abfss://silver@store.dfs.core.windows.net/orders/")

# Delta
df.write.format("delta").mode("append") \
.option("mergeSchema","true") \
.partitionBy("date") \
.save("abfss://gold@store.dfs.core.windows.net/orders/")

# Write modes:
# overwrite : replace all data
# append : add new data
# ignore : skip if data exists
# errorIfExists : raise error if exists (default)
```

Spark SQL — Using SQL on DataFrames

Register a DataFrame as a temp view and query it with SQL.

Code:

```
df.createOrReplaceTempView("orders")
df.createOrReplaceGlobalTempView("orders_global") # cross-session

result = spark.sql(
"SELECT customer_id, COUNT(*) AS order_count, SUM(amount) AS total_spend "
"FROM orders WHERE order_date >= '2024-01-01' "
"GROUP BY customer_id HAVING COUNT(*) > 5"
)
```

3.8 Partitioning & Bucketing

Repartition vs Coalesce

repartition(n) — full shuffle, increases or decreases partitions. coalesce(n) — no full shuffle, only decreases partitions (more efficient for reducing).

Code:

```
df.repartition(100) # full shuffle to 100 partitions
df.repartition(10, col("country")) # shuffle by column (co-locate same country)
df.coalesce(4) # reduce without full shuffle

# Check current partitions
print(df.rdd.getNumPartitions())
```

Partition Pruning

When a DataFrame is partitioned by a column and you filter on that column, Spark only reads the relevant partitions — dramatically reducing I/O.

Code:

```
# Data written partitioned by year/month
df.write.partitionBy("year","month").parquet("sales/")

# Reading with filter → only reads 2024/01 folder
spark.read.parquet("sales/").filter("year=2024 AND month=1")
```

Bucketing

Hash-partition data into a fixed number of files by a column. Enables bucket joins — joining two tables bucketed on the same key avoids a shuffle entirely.

Code:

```
df.write.bucketBy(64, "customer_id") \
.sortBy("customer_id") \
.saveAsTable("orders_bucketed")
df2.write.bucketBy(64, "customer_id") \
.sortBy("customer_id") \
.saveAsTable("customers_bucketed")
# Join — no shuffle!
spark.sql("SELECT * FROM orders_bucketed o JOIN customers_bucketed c ON o.customer_id = c.id")
```

3.9 Spark Optimization

Broadcast Join

Broadcast the smaller table to all executors so the larger table never needs to be shuffled. Automatically triggered for tables under spark.sql.autoBroadcastJoinThreshold (default 10MB).

Code:

```
from pyspark.sql.functions import broadcast
# Explicit broadcast
result = large_df.join(broadcast(small_lookup_df), "id")
# Increase threshold
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 100 * 1024 * 1024) # 100MB
```

Caching & Persisting

Store a DataFrame in memory (or memory+disk) so it is not recomputed on every action.

Code:

```
from pyspark import StorageLevel
df.cache() # MEMORY_AND_DISK
df.persist(StorageLevel.MEMORY_ONLY) # memory only
df.persist(StorageLevel.MEMORY_AND_DISK_SER) # serialised
# Unpersist when no longer needed (free memory)
df.unpersist()
# Check if cached
print(df.is_cached)
```

Skew Handling

Data skew occurs when one partition has far more data than others, causing one task to run much longer. Solutions: salting, AQE, repartition.

Code:

```
# Option 1: Adaptive Query Execution (Spark 3+)
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
# Option 2: Salting — add random suffix to join key
import pyspark.sql.functions as F
n_salt = 10
skewed_df = skewed_df.withColumn("salted_key",
concat(col("customer_id"), lit("_"),
(F.rand() * n_salt).cast("int")))
```

Adaptive Query Execution (AQE)

Spark 3+ feature that re-optimizes query plans at runtime based on actual data statistics — handles skew, coalesces shuffle partitions, switches join strategies automatically.

Code:

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

Catalyst Optimizer & Tungsten Engine

Catalyst is Spark's query optimizer — transforms logical plans to optimized physical plans (predicate pushdown, column pruning, join reordering). Tungsten handles low-level memory management and code generation for CPU efficiency.

Code:

```
# Explain plan to see Catalyst optimizations
df.explain() # simple
df.explain(mode="extended") # logical + optimized + physical plan
df.explain(mode="cost") # includes cost estimates
```

3.10 User Defined Functions (UDFs)

Python UDF

Custom Python functions applied per row. Slower than built-in functions due to serialization overhead.

Code:

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

def clean_phone(phone):
if phone is None:
return None
return ''.join(filter(str.isdigit, phone))

clean_phone_udf = udf(clean_phone, StringType())
df.withColumn("phone_clean", clean_phone_udf(col("phone")))

# Decorator syntax
@udf(returnType=StringType())
def mask_email(email):
if not email: return None
user, domain = email.split("@")
return user[:2] + "***@" + domain
```

Pandas UDF (Vectorized UDF)

UDFs that operate on Pandas Series/DataFrames using Apache Arrow for fast serialization. Much faster than row-by-row Python UDFs.

Code:

```
from pyspark.sql.functions import pandas_udf
import pandas as pd
@pandas_udf("double")
def normalize(series: pd.Series) -> pd.Series:
return (series - series.mean()) / series.std()
df.withColumn("salary_norm", normalize(col("salary")))
# Group Map UDF — apply function per group
from pyspark.sql.functions import PandasUDFType
@pandas_udf(df.schema, PandasUDFType.GROUPED_MAP)
def demean(pdf):
pdf["salary"] = pdf["salary"] - pdf["salary"].mean()
return pdf
df.groupby("department").apply(demean)
```

3.11 Structured Streaming

Reading a Stream

Code:

```
# Read from Event Hubs
ehConf = {
"eventhubs.connectionString": sc._jvm.org.apache.spark.eventhubs \
.EventHubsUtils.encrypt(connStr)
}
df_stream = spark.readStream.format("eventhubs").options(**ehConf).load()
# Read from Kafka
df_stream = spark.readStream.format("kafka") \
.option("kafka.bootstrap.servers","broker:9092") \
.option("subscribe","orders-topic") \
.load()
# Read from file folder (auto-detect new files)
df_stream = spark.readStream.format("parquet") \
.schema(schema) \
.load("abfss://landing@store.dfs.core.windows.net/events/")
```

Windowed Aggregations

Code:

```
from pyspark.sql.functions import window, col, sum
agg = df_stream \
.withWatermark("event_time", "10 minutes") \
.groupBy(
window(col("event_time"), "5 minutes"),
col("device_id")
).agg(sum("value").alias("total"))
# Output modes:
# append : only new rows (for no aggregation or with watermark)
# update : updated rows only
```

```
# complete: full result table every trigger
```

Writing a Stream

Code:

```
# Write to Delta table
query = agg.writeStream \
.format("delta") \
.outputMode("update") \
.option("checkpointLocation", "abfss://chk@store.dfs.core.windows.net/stream1/") \
.trigger(processingTime="30 seconds") \
.start("abfss://gold@store.dfs.core.windows.net/device_agg/")

query.awaitTermination()
query.stop()
```

4. Delta Lake — Complete Deep Dive

4.1 Core Concepts

What is Delta Lake?

An open-source storage layer on top of Parquet files that adds ACID transactions, versioning, schema enforcement, and scalable metadata. Every write is recorded in a _delta_log/ transaction log (JSON files).

Code:

```
-- Directory structure:
delta/orders/
_delta_log/
00000000000000000000.json <- version 0 (CREATE)
00000000000000000001.json <- version 1 (INSERT)
00000000000000000002.json <- version 2 (UPDATE)
00000000000000000010.checkpoint.parquet
part-00000-xxx.snappy.parquet
part-00001-xxx.snappy.parquet
```

ACID Transactions

Atomicity: all-or-nothing writes. Consistency: schema enforced. Isolation: concurrent writes are serialized. Durability: committed data survives failures.

Code:

```
-- Delta uses Optimistic Concurrency Control (OCC):
-- Writer reads current version, makes changes, tries to commit.
-- If another writer committed first, retries or raises conflict.

-- Check table history
from delta.tables import DeltaTable
dt = DeltaTable.forPath(spark, "delta/orders/")
dt.history().show(truncate=False)
```

4.2 Core Operations

Create Delta Table

Code:

```
# Write as Delta
df.write.format("delta").mode("overwrite").save("delta/orders/")

# As managed Spark table (metastore)
df.write.format("delta").mode("overwrite").saveAsTable("orders")

# Using SQL
spark.sql("CREATE TABLE IF NOT EXISTS sales "
"(id BIGINT, amount DOUBLE, dt DATE) "
"USING DELTA PARTITIONED BY (dt) "
"LOCATION 'abfss://gold@store.dfs.core.windows.net/sales/'")
```

Read Delta Table

Code:

```
# By path
df = spark.read.format("delta").load("delta/orders/")

# By table name
df = spark.table("orders")

# Specific version
df = spark.read.format("delta").option("versionAsOf", 5).load("delta/orders/")

# Specific timestamp
df = spark.read.format("delta").option("timestampAsOf","2024-06-01").load("delta/orders/")
```

MERGE — Upsert / SCD Type 1 & 2

Atomically insert, update, or delete rows based on a matching condition. The most important Delta operation for CDC and SCD patterns.

Code:

```
from delta.tables import DeltaTable

dt = DeltaTable.forPath(spark, "delta/customers/")

dt.alias("tgt").merge(
updates_df.alias("src"),
"tgt.customer_id = src.customer_id"
) \
.whenMatchedUpdate(set={
"email": "src.email",
"city": "src.city",
"updated_at": "src.updated_at"
}) \
.whenNotMatchedInsert(values={
"customer_id": "src.customer_id",
"name": "src.name",
"email": "src.email",
"city": "src.city",
"created_at": "src.created_at",
"updated_at": "src.updated_at"
}) \
.whenMatchedDelete(condition="src.is_deleted = true") \
.execute()
```

UPDATE & DELETE

Code:

```
from delta.tables import DeltaTable

dt = DeltaTable.forPath(spark, "delta/employees/")

# Update
dt.update(
condition="department = 'Engineering'",
set={"salary": "salary * 1.10"}
)

# Delete
dt.delete("is_active = false AND last_login < '2022-01-01'")
```

Time Travel

Code:

```
# Read old version
spark.read.format("delta").option("versionAsOf", 3).load("delta/orders/")
spark.read.format("delta").option("timestampAsOf","2024-01-15
10:00:00").load("delta/orders/")
# Restore table to previous version
from delta.tables import DeltaTable
dt = DeltaTable.forPath(spark, "delta/orders/")
dt.restoreToVersion(3)
dt.restoreToTimestamp("2024-01-15 10:00:00")
# View full history
dt.history().select("version","timestamp","operation","operationParameters").show()
```

Schema Enforcement & Evolution

Code:

```
# Schema enforcement (default) — rejects writes with wrong schema
df_wrong.write.format("delta").mode("append").save("delta/orders/")
# Raises AnalysisException if schema doesn't match
# Schema evolution — allow new columns
df_new.write.format("delta") \
.option("mergeSchema","true") \
.mode("append").save("delta/orders/")
# Override schema entirely
df.write.format("delta") \
.option("overwriteSchema","true") \
.mode("overwrite").save("delta/orders/")
```

OPTIMIZE & ZORDER

OPTIMIZE compacts small files into larger ones. ZORDER co-locates related data in the same files based on specified columns — dramatically reduces I/O for filtered queries.

Code:

```
from delta.tables import DeltaTable
dt = DeltaTable.forPath(spark, "delta/orders/")
dt.optimize().executeCompaction() # compact small files
dt.optimize().executeZOrderBy("customer_id","order_date") # ZOrder + compact
# Or via SQL
spark.sql("OPTIMIZE delta.`delta/orders/` ZORDER BY (customer_id, order_date)")
# VACUUM — remove old files no longer referenced (default 7-day retention)
dt.vacuum() # uses default 168h retention
dt.vacuum(retentionHours=24)
```

Change Data Feed (CDF)

Delta can expose row-level changes (insert/update/delete) as a stream — useful for CDC pipelines.

Code:

```
# Enable CDF on table
spark.sql("ALTER TABLE orders SET TBLPROPERTIES (delta.enableChangeDataFeed = true)")

# Read changes
changes = spark.read.format("delta") \
.option("readChangeFeed","true") \
.option("startingVersion", 5) \
.table("orders")

changes.show()
# Columns: _change_type (insert/update_preimage/update_postimage/delete)
# _commit_version, _commit_timestamp
```

5. Azure Data Factory — Complete Guide

5.1 Core Components

Pipeline

A logical grouping of activities that together perform a unit of work. A pipeline can contain one or many activities and can be triggered or called by other pipelines.

Code:

```
-- Pipeline concepts:
Activities : Steps (Copy, Dataflow, Notebook, StoredProc, Web, etc.)
Parameters : Dynamic values passed at runtime
Variables : Mutable values within a pipeline run
Annotations : Tags for organizing pipelines
```

Copy Activity

The most common ADF activity — moves data from a source to a sink. Supports 90+ connectors including SQL Server, Oracle, SAP, ADLS, Blob, Snowflake, REST APIs.

Code:

```
-- Copy Activity key settings:
Source : SQL Server table with query filter
Sink : ADLS Gen2 Parquet sink
Mapping : Column name/type mapping
Settings : Degree of parallelism, fault tolerance
Staging : PolyBase staging via Blob for Synapse loads
```

Linked Services

Connection definitions — like connection strings. Define WHERE to connect (server, auth, URL).

Code:

```
-- Common linked services:
Azure SQL Database : server + db + Managed Identity/password
ADLS Gen2 : account URL + Managed Identity
Azure Databricks : workspace + cluster/instance pool
Azure Key Vault : vault URL (for secret references)
REST : base URL + auth headers
On-premises SQL : via Self-Hosted Integration Runtime
```

Integration Runtime (IR)

The compute infrastructure for ADF. Three types: Azure IR (cloud), Self-Hosted IR (on-prem/VNet), Azure-SSIS IR (run SSIS packages).

Code:

```
-- Azure IR : default, serverless, auto-scales, public network
-- Self-Hosted IR : install agent on on-prem or VNet machine
-- needed for: on-prem SQL, Oracle, SAP, private VNet
-- Azure-SSIS IR : dedicated VM cluster to run SSIS packages
```

Triggers

Define when a pipeline runs. Four types: Schedule, Tumbling Window, Storage Event, Custom Event.

Code:

```
-- Schedule Trigger : cron expression (e.g. daily 2am)
-- Tumbling Window : fixed non-overlapping intervals, carries window start/end
-- supports retry & dependency on previous window
-- Storage Event : fires when blob is created/deleted in ADLS
-- Custom Event : fires on Azure Event Grid topic messages
```

5.2 Key Activities

ForEach Activity

Iterates over a collection (array parameter) and runs a set of inner activities for each item. Can run sequentially or in parallel.

Code:

```
-- Use case: loop over a list of tables and copy each one
-- Pipeline parameter: table_list = ["orders","customers","products"]
-- ForEach item: @item()
-- Inner Copy Activity source table: @item()
```

If Condition & Switch Activities

Branching logic in pipelines.

Code:

```
-- If Condition:
-- Expression: @equals(pipeline().parameters.env, 'prod')
-- True branch : run prod pipeline
-- False branch : run dev pipeline

-- Switch:
-- On: @pipeline().parameters.region
-- Cases: US, EU, APAC -> different linked services
```

Lookup & GetMetadata Activities

Code:

```
-- Lookup: execute a SQL query and pass results to next activity
-- e.g. SELECT MAX(updated_at) AS watermark FROM orders
-- Output: @activity('LookupWatermark').output.firstRow.watermark

-- GetMetadata: get properties of a file/folder in ADLS
-- Field list: exists, itemName, lastModified, size, childItems
-- e.g. check if file exists before copying
```

Dataflow Activity (Mapping Data Flows)

Visual ETL transformations that run on Spark clusters behind the scenes. No-code transformations: join, aggregate, pivot, conditional split, flatten, etc.

Code:

```
-- Common Dataflow transformations:
Source : read from linked service
Filter : row-level filter
Select : column selection/rename
Derived Column : add/compute columns
Aggregate : groupby + aggregations
Join : merge two streams
Conditional Split: route rows to different streams
Sink : write to target
```

Notebook & Databricks Activities

Code:

```
-- Azure Databricks Notebook Activity:
-- Linked Service: Databricks workspace
-- Notebook path: /Shared/ETL/process_orders
-- Base parameters: {"env": "prod", "date": "@pipeline().parameters.run_date"}
-- Cluster: existing interactive OR new job cluster
```

5.3 Dynamic Content & Expressions

ADF Expressions

ADF uses an expression language for dynamic values — functions, system variables, parameter references.

Code:

```
-- System variables
@pipeline().RunId
@pipeline().TriggerTime
@pipeline().parameters.env

-- String functions
@concat(pipeline().parameters.base_path, '/', formatDateTime(utcnow(),'yyyy/MM/dd'))

-- Conditional
@if(equals(pipeline().parameters.env,'prod'), 'prod-server', 'dev-server')

-- Date functions
@formatDateTime(adddays(utcnow(),-1), 'yyyy-MM-dd') -- yesterday
@startOfMonth(utcnow())
```

5.4 CI/CD for ADF

Git Integration & CI/CD Flow

Code:

```
-- 1. Connect ADF Studio to Azure DevOps Git repo
-- 2. Develop pipelines in feature branches
-- 3. Collaborate branch = main (published from here)
-- 4. Publish generates ARM templates in adf_publish branch
-- 5. Azure DevOps Release Pipeline deploys ARM to prod

-- ARM template files:
ARMTemplateForFactory.json -- all resources
ARMTemplateParametersForFactory.json -- parameter file (override per env)
```

6. Azure Synapse Analytics

Dedicated SQL Pool (formerly SQL DW)

A provisioned MPP (Massively Parallel Processing) data warehouse. Data is distributed across 60 compute nodes. Uses PolyBase for external data loading.

Code:

```
-- Distribution strategies:
HASH(column) : rows with same hash land on same node -> fast joins on that key
ROUND_ROBIN : rows spread evenly -> good for staging tables
REPLICATE : full copy on all 60 nodes -> good for small dimension tables (<2GB)
-- Index types:
Clustered Columnstore Index (CCI) : default, best for analytics (columnar, compressed)
Clustered Index : row-based, good for OLTP-style lookups
Heap : no index, fastest for staging loads
```

Serverless SQL Pool

Query data directly in ADLS/Blob using T-SQL without loading it. Pay per TB scanned. No infrastructure to manage.

Code:

```
-- Query Parquet directly
SELECT TOP 100 *
FROM OPENROWSET(
BULK 'https://mystorage.dfs.core.windows.net/silver/orders/**/*.parquet',
FORMAT = 'PARQUET'
) AS r;
-- Create external table over Parquet
CREATE EXTERNAL TABLE orders (
id INT,
amount FLOAT,
order_dt DATE
)
WITH (
LOCATION = 'silver/orders/',
DATA_SOURCE = adls_silver,
FILE_FORMAT = parquet_format
);
```

Apache Spark Pool in Synapse

Managed Spark clusters inside Synapse workspace. Auto-pause/resume, integrates with Synapse pipelines, native Delta Lake support.

Code:

```
-- Synapse Spark notebooks can read Delta tables registered in Synapse catalog
df = spark.read.synapsesql("synapse_pool.dbo.dim_product")
df.write.synapsesql("synapse_pool.dbo.fact_sales", mode="overwrite")
```

PolyBase & COPY INTO

Fast bulk load mechanism to get data from ADLS/Blob into Dedicated SQL Pool.

Code:

```
-- COPY INTO (simpler, recommended)
COPY INTO dbo.fact_sales
FROM 'https://mystorage.dfs.core.windows.net/gold/sales/*.parquet'
WITH (
FILE_TYPE = 'PARQUET',
CREDENTIAL = (IDENTITY = 'Managed Identity')
);

-- PolyBase external table approach
CREATE EXTERNAL TABLE ext_sales (...)
WITH (LOCATION='gold/sales/', DATA_SOURCE=..., FILE_FORMAT=...);

INSERT INTO dbo.fact_sales SELECT * FROM ext_sales;
```

7. Azure Data Lake Storage Gen2

Hierarchical Namespace (HNS)

Enables true directory semantics — atomic rename/move of directories (O(1) cost). Without HNS, renaming a directory requires copying all blobs and deleting originals (O(n) cost).

Code:

```
-- ABFS URI format:
abfss://@.dfs.core.windows.net/
-- Example:
abfss://gold@myadls.dfs.core.windows.net/sales/year=2024/month=01/
```

Access Control — ACLs vs RBAC

RBAC controls access at the storage account or container level. POSIX-style ACLs (Access Control Lists) provide fine-grained file/folder-level permissions.

Code:

```
-- RBAC roles for ADLS:
Storage Blob Data Reader : read any container
Storage Blob Data Contributor : read + write + delete
Storage Blob Data Owner : full + manage ACLs
-- ACL permissions: r (read), w (write), x (execute/traverse)
-- Set via Azure CLI:
az storage fs access set \
--acl "user:object-id:rwx" \
--path "silver/orders" \
--file-system silver \
--account-name myadls
```

Medallion Architecture on ADLS

Code:

```
containers/folders:
bronze/ <- raw ingested data (immutable, append-only)
raw/source_name/yyyy/mm/dd/filename.json
silver/ <- cleaned, validated, enriched
orders/year=2024/month=01/
gold/ <- business aggregates, star schema
fact_sales/ dim_product/ dim_customer/
```

SAS Tokens & Service Principals

Code:

```
-- SAS Token (Shared Access Signature): time-limited URL with embedded permissions
-- Use for: temporary access, external partner sharing

-- Service Principal: app registration in Azure AD
-- Grant RBAC role to service principal:
az role assignment create \
--assignee \
--role "Storage Blob Data Contributor" \
--scope "/subscriptions/.../storageAccounts/myadls"

-- Authenticate in PySpark:
spark.conf.set("fs.azure.account.auth.type.myadls.dfs.core.windows.net", "OAuth")
spark.conf.set("fs.azure.account.oauth.provider.type...", "ClientCredsTokenProvider")
spark.conf.set("fs.azure.account.oauth2.client.id...", "")
spark.conf.set("fs.azure.account.oauth2.client.secret...", "")
spark.conf.set("fs.azure.account.oauth2.client.endpoint...", "")
```

8. Azure Databricks

Cluster Types

Code:

```
-- All-Purpose Cluster : interactive, collaborative, persistent
-- Good for: notebooks, development, exploration
-- Expensive — terminate when not in use
-- Job Cluster : ephemeral, spun up for one job, terminated after
-- Good for: production pipelines, cost-efficient
-- Instance Pools : pre-allocated VMs — clusters start faster
-- Attach job clusters to pool for fast startup
```

Databricks File System (DBFS)

Abstraction layer over Azure Blob/ADLS — allows mount points so you can access ADLS paths using /mnt/ shortcuts instead of full abfss:// URIs.

Code:

```
# Mount ADLS Gen2
configs = {
"fs.azure.account.auth.type": "OAuth",
"fs.azure.account.oauth.provider.type":
"org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider",
"fs.azure.account.oauth2.client.id": dbutils.secrets.get("scope","client-id"),
"fs.azure.account.oauth2.client.secret": dbutils.secrets.get("scope","client-secret"),
"fs.azure.account.oauth2.client.endpoint": "https://login.microsoftonline.com//oauth2/token"
}
dbutils.fs.mount(
source = "abfss://silver@myadls.dfs.core.windows.net/",
mount_point = "/mnt/silver",
extra_configs = configs
)
# Now access via mount:
df = spark.read.parquet("/mnt/silver/orders/")
```

dbutils — Databricks Utilities

Code:

```
# File operations
dbutils.fs.ls("/mnt/silver/")
dbutils.fs.cp("/mnt/bronze/file.csv", "/mnt/silver/file.csv")
dbutils.fs.rm("/mnt/silver/old/", recurse=True)
dbutils.fs.mkdirs("/mnt/silver/new_folder/")

# Secrets
dbutils.secrets.get(scope="my-scope", key="db-password")
dbutils.secrets.listScopes()

# Widgets (notebook parameters)
dbutils.widgets.text("run_date", "2024-01-01")
run_date = dbutils.widgets.get("run_date")

# Notebook utilities
dbutils.notebook.run("./child_notebook", timeout_seconds=600,
arguments={"env": "prod"})
dbutils.notebook.exit("SUCCESS")
```

Delta Live Tables (DLT)

Declarative framework for building reliable Delta pipelines. Define tables with @dlt.table decorator, DLT manages dependencies, scheduling, error handling.

Code:

```
import dlt
from pyspark.sql.functions import col

@dlt.table(name="bronze_orders", comment="Raw orders from source")
def bronze_orders():
return spark.read.format("json").load("/mnt/landing/orders/")

@dlt.table(name="silver_orders", comment="Cleaned orders")
@dlt.expect("valid_amount", "amount > 0")
@dlt.expect_or_drop("non_null_id", "id IS NOT NULL")
def silver_orders():
return (dlt.read("bronze_orders")
.filter(col("status") != "CANCELLED")
.withColumn("amount_usd", col("amount") / col("fx_rate")))
```

Unity Catalog

Databricks' unified governance layer — central metastore for all workspaces, fine-grained access control, data lineage, and auditing.

Code:

```
-- 3-level namespace: catalog.schema.table
SELECT * FROM prod_catalog.silver.orders;

-- Grant permissions
GRANT SELECT ON TABLE prod_catalog.silver.orders TO `data-analyst-group`;
GRANT MODIFY ON SCHEMA prod_catalog.gold TO `data-engineer-group`;

-- Data lineage tracked automatically for all Delta operations
```

9. Event Hubs & Stream Analytics

Event Hubs Architecture

Event Hubs is a Kafka-compatible streaming platform. Messages are persisted in partitions with configurable retention (1-90 days).

Code:

```
Namespace
Event Hub 1 (topic)
Partition 0 [msg0, msg1, msg2, ...]
Partition 1 [msg0, msg1, msg2, ...]
Partition 2 [msg0, msg1, msg2, ...]
Event Hub 2
...
-- Consumer Group: independent read cursor per application
-- Throughput Units (TU): 1 TU = 1 MB/s ingress, 2 MB/s egress
-- Capture: auto-save raw events to ADLS Gen2 as Avro/Parquet
```

Sending Events to Event Hubs

Code:

```
from azure.eventhub import EventHubProducerClient, EventData
import json
producer = EventHubProducerClient.from_connection_string(
conn_str="",
eventhub_name="orders"
)
with producer:
batch = producer.create_batch()
for order in orders:
batch.add(EventData(json.dumps(order)))
producer.send_batch(batch)
```

Stream Analytics Query

SQL-like language for real-time stream processing — reads from Event Hubs, applies windowed aggregations, writes to output sinks.

Code:

```
-- Count orders per product every 5 minutes
SELECT
product_id,
COUNT(*) AS order_count,
SUM(amount) AS total_amount,
System.Timestamp() AS window_end
INTO output_cosmos
FROM orders_input TIMESTAMP BY order_time
GROUP BY product_id, TumblingWindow(minute, 5);
-- Alert when avg temperature exceeds threshold (sliding window)
SELECT device_id, AVG(temperature) AS avg_temp
FROM iot_input TIMESTAMP BY event_time
GROUP BY device_id, SlidingWindow(minute, 5)
HAVING AVG(temperature) > 80;
```

10. Data Modeling & Warehouse Design

10.1 Star & Snowflake Schema

Star Schema

A fact table at the center connected to denormalized dimension tables via foreign keys. Optimized for read performance — requires few joins, easy for BI tools.

Code:

```
FactSales
sale_id BIGINT (PK surrogate)
date_id INT (FK -> DimDate)
product_id INT (FK -> DimProduct)
customer_id INT (FK -> DimCustomer)
store_id INT (FK -> DimStore)
quantity INT
unit_price DECIMAL
total_amt DECIMAL
DimProduct
product_id INT (PK)
product_name, category, sub_category, brand, color, size
DimDate
date_id INT (PK), full_date, year, quarter, month,
month_name, week, day_of_week, is_weekend, is_holiday
```

Snowflake Schema

Dimension tables are normalized — sub-dimensions broken into their own tables. Saves storage but requires more joins. Less common in modern cloud DWH.

Code:

```
DimProduct -> DimSubCategory -> DimCategory -> DimDepartment
-- Each is a separate table with FK relationships
-- Use when: dimension tables are very large, storage is critical
```

Galaxy Schema (Fact Constellation)

Multiple fact tables sharing dimension tables. Common in enterprise DWH.

Code:

```
FactSales --.
+--> DimDate, DimProduct, DimCustomer (shared dims)
FactReturns -'
```

10.2 Slowly Changing Dimensions (SCD)

SCD Type 1 — Overwrite

Simply overwrite the old value. No history preserved. Use when history doesn't matter.

Example:

Customer changes phone number — old number is lost.

Code:

```
UPDATE DimCustomer
SET phone = '555-9999', updated_at = GETDATE()
WHERE customer_id = 101;
```

SCD Type 2 — Add New Row

Insert a new row with new values, mark old row as expired. Full history preserved. Requires surrogate key, effective_date, expiry_date, and is_current flag.

Example:

Customer moves from NYC to LA — both records kept.

Code:

```
-- Old row
(surrogate=1, customer_id=101, city='NYC', eff='2020-01-01', exp='2024-06-14', is_current=0)
-- New row
(surrogate=2, customer_id=101, city='LA', eff='2024-06-15', exp='9999-12-31', is_current=1)
```

```
-- PySpark MERGE for SCD2
from delta.tables import DeltaTable
dt = DeltaTable.forPath(spark, "delta/dim_customer/")
dt.alias("tgt").merge(updates.alias("src"),
"tgt.customer_id = src.customer_id AND tgt.is_current = 1"
).whenMatchedUpdate(
condition="tgt.city <> src.city",
set={"is_current":"0", "expiry_date":"src.effective_date"}
).whenNotMatchedInsertAll().execute()
```

SCD Type 3 — Add Column

Add a 'previous_value' column. Only tracks one level of history.

Code:

```
-- Table has: city, previous_city, city_changed_date
UPDATE DimCustomer
SET previous_city = city,
city = 'LA',
city_changed_date = GETDATE()
WHERE customer_id = 101;
```

10.3 Surrogate Keys & Natural Keys

Surrogate vs Natural Keys

Natural key: business identifier from source (order_number, customer_email). Surrogate key: system-generated integer, independent of business logic. Always use surrogate keys in dimension tables — protects against source system changes.

Code:

```
-- Surrogate key generation in PySpark
from pyspark.sql.functions import monotonically_increasing_id
df.withColumn("surrogate_key", monotonically_increasing_id())
-- Or with row_number for deterministic keys:
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number
w = Window.orderBy("customer_id")
df.withColumn("dim_key", row_number().over(w))
```

10.4 Medallion Architecture (Deep Dive)

Bronze Layer

Raw, unmodified data exactly as received from source. Append-only. Acts as the system of record — all reprocessing starts from here.

Code:

```
-- Bronze conventions:
-- Store raw JSON/CSV/Avro/XML as-is
-- Add metadata columns: _ingested_at, _source_file, _pipeline_run_id
-- Partition by ingestion date, not business date
-- Never transform or clean data in bronze
df_raw.withColumn("_ingested_at", current_timestamp()) \
.withColumn("_source", lit("orders_api")) \
.write.format("delta").mode("append") \
.partitionBy("_ingest_date") \
.save("abfss://bronze@store.dfs.core.windows.net/orders/")
```

Silver Layer

Cleaned, validated, deduplicated, and conformed data. Schema applied, data types corrected, business rules applied. Still detailed/atomic.

Code:

```
-- Silver transformations:
-- 1. Cast to correct types
-- 2. Standardize nulls / missing values
-- 3. Deduplicate
-- 4. Apply data quality rules (reject/flag bad rows)
-- 5. Enrich (join lookups)
-- 6. Rename to business names
df_silver = (df_bronze
.dropDuplicates(["order_id"])
.filter(col("amount") > 0)
.withColumn("order_date", to_date(col("order_date_str"), "yyyy-MM-dd"))
.withColumn("customer_id", col("cust_id").cast(IntegerType()))
.drop("_source_raw_field"))
```

Gold Layer

Business-level aggregates and conformed dimensional models. Purpose-built for specific use cases: BI reports, ML features, APIs.

Code:

```
-- Gold examples:
-- fact_sales: daily sales per product per store
-- daily_active_users: count per day per platform
-- customer_360: enriched customer profile
-- product_inventory_snapshot: daily stock levels
```

```
gold_df = (silver_orders
.join(silver_products, "product_id")
.join(dim_date, "order_date")
.groupBy("date_key","product_key","store_key")
.agg(
sum("quantity").alias("total_qty"),
sum("amount").alias("total_revenue"),
countDistinct("customer_id").alias("unique_customers")
))
```

10.5 Data Quality

Great Expectations — Data Quality Framework

Open-source library for defining, testing, and documenting data quality expectations.

Code:

```
import great_expectations as ge
```

```
df_ge = ge.dataset.SparkDFDataset(df)
df_ge.expect_column_to_exist("order_id")
df_ge.expect_column_values_to_not_be_null("order_id")
df_ge.expect_column_values_to_be_between("amount", 0, 1000000)
df_ge.expect_column_values_to_be_unique("order_id")
df_ge.expect_column_values_to_match_regex("email",
r'^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+[.][a-zA-Z0-9-.]+$')
results = df_ge.validate()
```

11. Real-Time & Streaming Data

11.1 Streaming Concepts

Batch vs Micro-batch vs True Streaming

Code:

```
Batch : process large historical data on a schedule (e.g. nightly)
Tools: ADF Copy, Spark batch jobs

Micro-batch : process small windows of data every N seconds/minutes
Tools: Spark Structured Streaming (trigger interval)

True Streaming: process each event as it arrives (sub-second)
Tools: Apache Flink, Azure Stream Analytics
```

Watermarks

A threshold that tells the streaming engine how late data can arrive and still be included in a window. Events arriving later than the watermark are dropped.

Example:

Events can arrive up to 10 minutes late — watermark = 10 minutes.

Code:

```
df_stream.withWatermark("event_time", "10 minutes") \
.groupBy(window("event_time", "5 minutes"), "device_id") \
.count()

-- Without watermark: state grows unboundedly (memory leak risk)
-- With watermark: old state is dropped after threshold
```

Checkpointing

Saves the streaming query state (offsets, aggregation state) to durable storage. Enables fault-tolerance — query can resume from exact position after failure.

Code:

```
query = df.writeStream \
.format("delta") \
.option("checkpointLocation", "abfss://chk@store.dfs.core.windows.net/orders_stream/") \
.start("abfss://gold@store.dfs.core.windows.net/orders/")
-- Checkpoint stores:
-- offsets/ : source read positions
-- commits/ : committed batch IDs
-- state/ : aggregation state (if any)
```

Window Types

Code:

```
-- Tumbling Window: fixed, non-overlapping
[0:00-0:05] [0:05-0:10] [0:10-0:15]
TumblingWindow(minute, 5)

-- Hopping/Sliding Window: overlapping, slides by step
[0:00-0:05] [0:01-0:06] [0:02-0:07]
SlidingWindow(minute, 5, 1) -- 5 min window, 1 min hop

-- Session Window: closes after inactivity gap
[events...][gap > 10min][events...]
```

```
SessionWindow(minute, 10)
```

Lambda Architecture vs Kappa Architecture

Code:

```
Lambda Architecture:
Batch Layer : process all historical data (accurate, slow)
Speed Layer : process recent data in real-time (fast, approximate)
Serving Layer: merge batch + speed layer results
Downside: two codebases to maintain

Kappa Architecture:
Single streaming pipeline handles both real-time AND reprocessing
Reprocess by replaying the event log from the beginning
Simpler: one codebase, one system
Preferred in modern cloud architectures with Event Hubs + Delta
```

12. Security & Governance

12.1 Azure Identity & Access

Azure Active Directory (Entra ID)

Microsoft's cloud identity platform. All Azure resources use it for authentication. Users, groups, service principals, and managed identities are all Entra ID objects.

Code:

```
-- Key concepts:
Tenant : Organization's AAD instance (unique directory)
User : Human identity with email/password
Group : Collection of users — assign roles to groups
App Registration : Identity for applications (has client ID + secret)
Service Principal : Instance of an app registration in a tenant
```

RBAC — Role-Based Access Control

Assign roles at scope levels: Management Group > Subscription > Resource Group > Resource. Child scopes inherit parent assignments.

Code:

```
-- Assign role via Azure CLI
az role assignment create \
--assignee OBJECT_ID \
--role "Storage Blob Data Contributor" \
--scope
"/subscriptions/SUB_ID/resourceGroups/RG_NAME/providers/Microsoft.Storage/storageAccounts/"
-- Common data engineering roles:
Storage Blob Data Reader : read ADLS
Storage Blob Data Contributor : read+write ADLS
Synapse Administrator : full Synapse
Data Factory Contributor : manage ADF
Key Vault Secrets User : read secrets
```

Managed Identity

Azure-managed service identity — no credentials to manage. System-assigned: tied to one resource, deleted with it. User-assigned: independent, can be shared across multiple resources.

Code:

```
-- Enable System-Assigned MI on ADF
-- Then grant it role on ADLS:
az role assignment create \
--assignee ADF_MI_OBJECT_ID \
--role "Storage Blob Data Contributor" \
--scope "/subscriptions/.../storageAccounts/myadls"
-- In ADF linked service: Authentication = Managed Identity
-- No passwords, no rotation needed
```

Azure Key Vault

Centralized secrets store — connection strings, passwords, certificates, keys. Integrate with ADF, Databricks, and code via SDK.

Code:

```
-- ADF: reference secret in linked service connection string
@Microsoft.KeyVault(SecretUri=https://myvault.vault.azure.net/secrets/db-pass/version)
-- Databricks: use secret scope backed by Key Vault
dbutils.secrets.get(scope="kv-scope", key="db-password")
-- Python SDK
from azure.keyvault.secrets import SecretClient
from azure.identity import DefaultAzureCredential
client = SecretClient("https://myvault.vault.azure.net", DefaultAzureCredential())
pwd = client.get_secret("db-password").value
```

12.2 Data Governance

Microsoft Purview — Data Catalog

Unified governance service: scan sources, auto-classify data, build lineage, create data dictionary.

Code:

```
-- Key capabilities:
Data Map : register and scan data sources (ADLS, SQL, Synapse, Power BI)
Data Catalog : search and discover assets by name, column, classification
Classifications: auto-tag sensitive data (SSN, email, credit card, GDPR)
Data Lineage : track data flow from source -> ADF -> Synapse -> Power BI
Glossary : define business terms and link to physical assets
Insights : dashboards on coverage, sensitivity, stewardship
```

Row-Level Security (RLS)

Restrict which rows a user can see in a table based on their identity.

Code:

```
-- SQL Server / Synapse RLS
CREATE SCHEMA Security;
CREATE FUNCTION Security.fn_securitypredicate(@Region AS VARCHAR(50))
RETURNS TABLE
AS RETURN SELECT 1 AS fn_result
WHERE @Region = USER_NAME() OR USER_NAME() = 'admin';
CREATE SECURITY POLICY RegionFilter
ADD FILTER PREDICATE Security.fn_securitypredicate(region) ON dbo.sales;
-- Databricks Unity Catalog RLS
CREATE ROW FILTER filter_by_region ON TABLE sales
USING (region = current_user());
```

Column-Level Security & Dynamic Data Masking

Code:

```
-- Column-level security: deny SELECT on specific columns
DENY SELECT ON dbo.employees(salary, ssn) TO analyst_role;

-- Dynamic Data Masking: mask data at query time without changing storage
ALTER TABLE customers
ALTER COLUMN email ADD MASKED WITH (FUNCTION = 'email()');
-- analyst sees: aXXX@XXXX.com instead of alice@example.com

ALTER TABLE employees
ALTER COLUMN salary ADD MASKED WITH (FUNCTION = 'random(1, 12)');
```

Private Endpoints & VNet Integration

Keep data traffic inside Azure private network — prevents exposure over public internet.

Code:

```
-- Private Endpoint: dedicated private IP for a service inside your VNet
-- ADLS Gen2 private endpoint -> traffic stays in VNet
-- Synapse private endpoint -> no public access needed

-- ADF Managed VNet: ADF runtime deployed inside a managed VNet
-- Use managed private endpoints to connect to ADLS, SQL, etc.
-- Required for: compliance, regulated industries

-- Self-Hosted IR: installed on on-prem or VNet VM
-- Bridges on-premises network to ADF
```

13. DevOps & CI/CD for Data Pipelines

13.1 Git Strategy for Data Teams

Branching Strategy

Code:

```
main : production-ready code, protected
develop : integration branch, all features merged here
feature/* : individual feature development (feature/add-orders-pipeline)
hotfix/* : urgent production fixes
release/* : release preparation

-- PR process:
-- feature/* -> develop (code review required)
-- develop -> main (release process)
```

ADF Git Integration

Code:

```
-- Setup in ADF Studio: Manage -> Git configuration
-- Repository type: Azure DevOps Git or GitHub
-- Collaboration branch: main (all devs merge here)
-- Publish branch: adf_publish (auto-generated ARM templates)
-- Root folder: /adf (optional, for mono-repo)

-- Workflow:
1. Create feature branch
2. Build/edit pipelines in ADF Studio
3. Save (commits JSON to feature branch)
4. PR to main -> code review
5. Merge to main
6. Click Publish -> generates ARM in adf_publish
7. Release pipeline deploys ARM to prod
```

13.2 Azure DevOps Pipelines

CI Pipeline — Build Validation

Code:

```
# azure-pipelines.yml (CI - runs on PR)
trigger:
branches:
include: [develop, main]

pool:
vmImage: ubuntu-latest

steps:
- task: UsePythonVersion@0
inputs:
versionSpec: '3.10'

- script: pip install pytest pyspark delta-spark
displayName: 'Install dependencies'

- script: pytest tests/ -v --junitxml=results.xml
displayName: 'Run unit tests'

- task: PublishTestResults@2
inputs:
testResultsFiles: results.xml
```

CD Pipeline — Deploy ADF ARM Template

Code:

```
# Release pipeline stage: Deploy to Production
- task: AzureResourceManagerTemplateDeployment@3
inputs:
deploymentScope: Resource Group
azureResourceManagerConnection: azure-prod-connection
subscriptionId: $(SUBSCRIPTION_ID)
action: Create Or Update Resource Group
resourceGroupName: rg-prod-data
location: East US
templateLocation: Linked artifact
csmFile: $(Pipeline.Workspace)/adf_publish/ARMTemplateForFactory.json
csmParametersFile: $(Pipeline.Workspace)/adf_publish/ARMTemplateParametersForFactory.json
overrideParameters: >
-factoryName adf-prod
-AzureDataLakeStorage_accountKey $(ADLS_KEY)
```

Unit Testing PySpark Transformations

Code:

```
import pytest
from pyspark.sql import SparkSession
from transformations import clean_orders # your module
@pytest.fixture(scope="session")
def spark():
return SparkSession.builder.master("local[2]").appName("test").getOrCreate()
def test_clean_orders_removes_nulls(spark):
data = [(1, "Alice", 100.0), (2, None, 50.0), (3, "Bob", -10.0)]
df = spark.createDataFrame(data, ["id","name","amount"])
result = clean_orders(df)
assert result.count() == 1
assert result.collect()[0]["name"] == "Alice"
def test_clean_orders_removes_negative_amounts(spark):
data = [(1, "Alice", -5.0), (2, "Bob", 100.0)]
df = spark.createDataFrame(data, ["id","name","amount"])
result = clean_orders(df)
assert result.count() == 1
```

13.3 Infrastructure as Code

Terraform for Azure Data Infrastructure

Code:

```
# main.tf
resource "azurerm_resource_group" "rg" {
name = "rg-${var.env}-data"
location = var.location
}
resource "azurerm_storage_account" "adls" {
name = "adls${var.env}${var.suffix}"
resource_group_name = azurerm_resource_group.rg.name
location = azurerm_resource_group.rg.location
account_tier = "Standard"
account_replication_type = "LRS"
is_hns_enabled = true # ADLS Gen2
}
resource "azurerm_data_factory" "adf" {
name = "adf-${var.env}"
resource_group_name = azurerm_resource_group.rg.name
location = azurerm_resource_group.rg.location
identity { type = "SystemAssigned" }
}
resource "azurerm_role_assignment" "adf_adls" {
scope = azurerm_storage_account.adls.id
role_definition_name = "Storage Blob Data Contributor"
principal_id = azurerm_data_factory.adf.identity[0].principal_id
}
```

Monitoring & Alerting

Code:

```
-- Azure Monitor Alert for ADF pipeline failure:
az monitor metrics alert create \
--name "ADF Pipeline Failure Alert" \
--resource-group rg-prod \
--scopes /subscriptions/.../factories/adf-prod \
--condition "count PipelineFailedRuns > 0" \
--window-size 5m \
--evaluation-frequency 1m \
--action-group /subscriptions/.../actionGroups/email-team

-- KQL query in Log Analytics for ADF runs:
ADFPipelineRun
| where Status == "Failed"
| project PipelineName, Start, End, ErrorMessage
| order by Start desc
```

14. Microsoft Fabric & DP-700

What is Microsoft Fabric?

An end-to-end analytics platform unifying data engineering, data warehousing, data science, real-time analytics, and Power BI in one SaaS product. Built on OneLake (one data lake for the whole org).

Code:

```
-- Key workloads:
Data Engineering : Lakehouse, Notebooks, Spark, Data Factory pipelines
Data Warehouse : SQL-based DWH with T-SQL
Data Science : ML experiments, model training
Real-Time Analytics: KQL databases, Eventstream
Power BI : embedded, unified with data
Data Activator : alerts and actions on data changes
```

OneLake

Single logical data lake for the entire organization — all Fabric items store data in OneLake in Delta-Parquet format. No data silos between workloads.

Code:

```
-- OneLake path structure:
OneLake///

-- Access from outside Fabric:
abfss://@onelake.dfs.fabric.microsoft.com//Tables/

-- All Fabric engines (SQL, Spark, Power BI) read the same Delta files
-- No data movement or duplication between workloads
```

Lakehouse

Combines a data lake (ADLS-backed file storage) with a SQL analytics endpoint. Tables are Delta format. Files section holds raw data. SQL endpoint auto-generated.

Code:

```
-- Fabric Lakehouse structure:
Tables/ <- Delta tables (queryable via SQL endpoint and Spark)
Files/ <- Unstructured files, raw landing zone

-- Create table from notebook
df.write.format("delta").saveAsTable("silver.orders")

-- Instantly queryable via SQL endpoint:
SELECT * FROM silver.orders WHERE order_date >= '2024-01-01'
```

Data Factory in Fabric

Next-gen ADF inside Fabric. Uses same pipeline/activity concepts but integrated with Fabric workspace. Supports Copy, Dataflow Gen2, and Notebook activities.

Code:

```
-- Key differences from ADF:
-- Dataflow Gen2: uses Power Query M formula engine (no-code ETL)
-- Pipelines natively aware of Lakehouse, Warehouse
-- Shortcuts: reference external data (ADLS, S3, GCS) without copying
-- Staging auto-configured for Lakehouse loads
```

Shortcuts

Virtual references to external data sources (ADLS Gen2, AWS S3, GCS, other Fabric workspaces) that appear as native Fabric tables/files — no data movement.

Code:

```
-- Create shortcut to ADLS Gen2 in Fabric UI:
-- Lakehouse -> New shortcut -> Azure Data Lake Storage Gen2
-- Specify: account URL, container, subpath
-- Result: appears as Files/ subfolder, queryable via Spark/SQL
-- Use case: access existing ADLS data without migration
```

KQL Database & Eventstream

KQL (Kusto Query Language) Database: optimized for time-series and log analytics. Eventstream: no-code real-time data ingestion pipeline.

Code:

```
-- Eventstream sources: Event Hubs, IoT Hub, Kafka, custom app
-- Eventstream destinations: KQL Database, Lakehouse, another Eventstream
-- KQL query example:
SensorReadings
| where Timestamp >= ago(1h)
| where Temperature > 80
| summarize avg_temp=avg(Temperature) by bin(Timestamp, 5m), DeviceId
| order by Timestamp desc
```

15. Certifications & Learning Roadmap

15.1 Certification Details

Exam	Level	Focus Areas	Duration
DP-900	Beginner	Core data concepts, relational/non-relational, analytics basics	45 min
AZ-900	Beginner	Azure fundamentals — good prereq before DP-203	45 min
DP-203	Associate	ADF, Synapse, Databricks, ADLS, Spark, streaming, security	120 min
DP-700	Associate	Microsoft Fabric, Lakehouse, KQL, Eventstream, DWH	100 min
DP-100	Associate	Azure ML, data science, model training (optional)	180 min

15.2 DP-203 Exam Topics Breakdown

Domain	Weight
Design & implement data storage	40-45%
Design & develop data processing	25-30%
Design & implement data security	10-15%
Monitor & optimize data storage and processing	10-15%

15.3 Step-by-Step Learning Roadmap

Weeks 1-2	**SQL Mastery**	Window functions, CTEs, complex joins, optimization, execution plans
Weeks 3-4	**Python + Pandas**	DataFrames, GroupBy, merge, file I/O, Azure SDK basics
Weeks 5-8	**PySpark Deep Dive**	All transformations, window functions, joins, UDFs, optimization, streaming
Weeks 9-10	**Delta Lake**	ACID, MERGE, time travel, OPTIMIZE, CDF, schema evolution
Weeks 11-12	**Azure Core Services**	ADLS Gen2, ADF pipelines, Synapse SQL pools, Azure SQL
Weeks 13-14	**Databricks**	Clusters, DBFS, dbutils, DLT, Unity Catalog
Weeks 15-16	**Security & Governance**	RBAC, Managed Identity, Key Vault, Purview, RLS, Private Endpoints
Weeks 17-18	**DevOps & Architecture**	CI/CD, Terraform, testing, medallion architecture, data modeling
Weeks 19-20	**Streaming & Exam Prep**	Event Hubs, Stream Analytics, Structured Streaming, practice exams

15.4 Key Resources

- Microsoft Learn: learn.microsoft.com — free official learning paths for DP-203, DP-700
- Azure Documentation: docs.microsoft.com/azure — official service docs

- Databricks Academy: academy.databricks.com — free Spark and Delta courses
- Delta Lake docs: delta.io/docs — official Delta Lake documentation
- PySpark docs: spark.apache.org/docs/latest/api/python — full API reference
- Practice exams: MeasureUp, Whizlabs, Microsoft's official practice assessments
- GitHub: search 'azure data engineering projects' — real-world project examples
- YouTube: Will Velida, Guy in a Cube, Pragmatic Works — ADF and Synapse tutorials

You now have everything you need. Build projects, practice daily, and you will land that Azure Data Engineer role.